Compiler Design (Finalterm Lab 1)

Task 1: Write a C++ program that takes the description of a **Deterministic Finite Automaton (DFA)** and one input string, then checks whether the string is **Accepted** or **Rejected** by the DFA.

Input format

- First line: two integers n and m
  - n = number of states, states are numbered from 0 to n-1
  - m = number of input symbols
- Next m lines: the input symbols
- Next line: integer t, the number of transitions
- Next t lines: current_state input_symbol next_state
- Next line: start state
- Next line: integer f, the number of final states
- Next line: f integers representing final states
- Last line: one input string

Output

Print:

- Accepted if the string is accepted by the DFA
- Rejected otherwise

Sample Input	*Sample Output*
3 2 a b 6 0 a 1 0 b 0 1 a 1 1 b 2 2 a 1 2 b 0 0 1 2 aab	Accepted

Task 2: Write a C++ program that takes the description of a DFA and first checks whether it is a **valid DFA**. If it is valid, then test multiple input strings and print the **state path** and final result for each string.

A DFA will be considered valid only if:

- every state number is within range
- every transition symbol belongs to the given alphabet
- there is not more than one transition for the same (state, symbol) pair
- the start state is valid
- every final state is valid
- for every state and every symbol, there is exactly one transition

Input format

Use the same DFA input format as Task 1, then:

- one line containing integer q, the number of test strings
- next q lines, one input string per line

Output

- If the DFA is invalid, print:
  - Invalid DFA
  - and the reason
- If the DFA is valid, print:
  - Valid DFA
  - then for each input string:
    - the visited path
    - Accepted or Rejected

Sample Input:	*Sample Output:*
3 2 a b 6 0 a 1 0 b 0 1 a 1 1 b 2 2 a 1 2 b 0 0 1 2 3 ab aab bbb	Valid DFA String: ab Path: 0 -> 1 -> 2 Accepted String: aab Path: 0 -> 1 -> 1 -> 2 Accepted String: bbb Path: 0 -> 0 -> 0 -> 0 Rejected